Assignment 4: Data Wrangling (Fall 2025)

Yitong Eric Su

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics on Data Wrangling

Directions

1. Rename this file `<FirstLast>_A04_DataWrangling.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change "Student Name" on line 3 (above) with your name.
3. Work through the steps, **creating code and output** that fulfill each instruction.
4. Be sure to **answer the questions** in this assignment document.
5. When you have completed the assignment, **Knit** the text and code into a single PDF file.
6. Ensure that code in code chunks does not extend off the page in the PDF.

Set up your session

1a. Load the `tidyverse`, `lubridate`, and `here` packages into your session.

1b. Check your working directory.

1c. Read in all four raw data files associated with the EPA Air dataset, being sure to set string columns to be read in a factors. See the README file for the EPA air datasets for more information (especially if you have not worked with air quality data previously).

2. Add the appropriate code to reveal the dimensions of the four datasets.

```
#1a load the packages
library(tidyverse)
library(lubridate)
library(here)

#1b check the working directory
here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
getwd()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

```
#1c  Read the EPA datasets
EPAair_O3_2018 <- read.csv(
  file = here("./Data/Raw/EPAair_O3_NC2018_raw.csv"), stringsAsFactors = TRUE)
EPAair_O3_2019 <- read.csv(
  file = here("./Data/Raw/EPAair_O3_NC2019_raw.csv"), stringsAsFactors = TRUE)
EPAair_PM25_2018 <- read.csv(
  file = here("./Data/Raw/EPAair_PM25_NC2018_raw.csv"), stringsAsFactors = TRUE)
EPAair_PM25_2019 <- read.csv(
  file = here("./Data/Raw/EPAair_PM25_NC2019_raw.csv"), stringsAsFactors = TRUE)

#2 check the dimensions of the four datasets
dim(EPAair_O3_2018)
```

```
## [1] 9737   20
```

```
dim(EPAair_O3_2019)
```

```
## [1] 10592    20
```

```
dim(EPAair_PM25_2018)
```

```
## [1] 8983   20
```

```
dim(EPAair_PM25_2019)
```

```
## [1] 8581   20
```

All four datasets should have the same number of columns but unique record counts (rows). Do your datasets follow this pattern?

Answer: Yes. All the four imported datasets have 20 columns and different record counts.

Wrangle individual datasets to create processed files.

3. Change the Date columns to be date objects.
4. Select the following columns: Date, DAILY_AQI_VALUE, Site.Name, AQS_PARAMETER_DESC, COUNTY, SITE_LATITUDE, SITE_LONGITUDE
5. For the PM2.5 datasets, fill all cells in AQS_PARAMETER_DESC with "PM2.5" (all cells in this column should be identical).
6. Save all four processed datasets in the Processed folder. Use the same file names as the raw files but replace "raw" with "processed".

```
#3 Change the date column to be date objects
EPAair_O3_2018$Date <- as.Date(EPAair_O3_2018$Date, format = "%m/%d/%y")
EPAair_O3_2019$Date <- as.Date(EPAair_O3_2019$Date, format = "%m/%d/%y")
EPAair_PM25_2018$Date <- as.Date(EPAair_PM25_2018$Date, format = "%m/%d/%y")
EPAair_PM25_2019$Date <- as.Date(EPAair_PM25_2019$Date, format = "%m/%d/%y")
```

```
#4 Select columns
EPAair_O3_2018_selected <- EPAair_O3_2018 %>%
  select(Date, DAILY_AQI_VALUE, Site.Name, AQS_PARAMETER_DESC,
         COUNTY, SITE_LATITUDE, SITE_LONGITUDE)
EPAair_O3_2019_selected <- EPAair_O3_2019 %>%
  select(Date, DAILY_AQI_VALUE, Site.Name, AQS_PARAMETER_DESC,
         COUNTY, SITE_LATITUDE, SITE_LONGITUDE)
EPAair_PM25_2018_selected <- EPAair_PM25_2018 %>%
  select(Date, DAILY_AQI_VALUE, Site.Name, AQS_PARAMETER_DESC,
         COUNTY, SITE_LATITUDE, SITE_LONGITUDE)
EPAair_PM25_2019_selected <- EPAair_PM25_2019 %>%
  select(Date, DAILY_AQI_VALUE, Site.Name, AQS_PARAMETER_DESC,
         COUNTY, SITE_LATITUDE, SITE_LONGITUDE)

#5 Fill cells in AQS_PARAMETER_DESC of the PM2.5 datasets
EPAair_PM25_2018_selected$AQS_PARAMETER_DESC <- "PM2.5"
EPAair_PM25_2019_selected$AQS_PARAMETER_DESC <- "PM2.5"

#6 save the processed datasets to the "processed" folder
write.csv(EPAair_O3_2018_selected, row.names = FALSE,
          here("Data", "Processed", "EPAair_O3_NC2018_processed.csv"))
write.csv(EPAair_O3_2019_selected, row.names = FALSE,
          here("Data", "Processed", "EPAair_O3_NC2019_processed.csv"))
write.csv(EPAair_PM25_2018_selected, row.names = FALSE,
          here("Data", "Processed", "EPAair_PM25_NC2018_processed.csv"))
write.csv(EPAair_PM25_2019_selected, row.names = FALSE,
          here("Data", "Processed", "EPAair_PM25_NC2019_processed.csv"))
```

Combine datasets

7. Combine the four datasets with `rbind`. Make sure your column names are identical prior to running this code.

8. Wrangle your new dataset with a pipe function (%>%) so that it fills the following conditions:

- Include only sites that the four data frames have in common:

"Linville Falls", "Durham Armory", "Leggett", "Hattie Avenue",
"Clemmons Middle", "Mendenhall School", "Frying Pan Mountain", "West Johnston Co.", "Garinger High School", "Castle Hayne", "Pitt Agri. Center", "Bryson City", "Millbrook School"

(the function `intersect` can figure out common factor levels - but it will include sites with missing site information, which you don't want...)

- Some sites have multiple measurements per day. Use the split-apply-combine strategy to generate daily means: group by date, site name, AQS parameter, and county. Take the mean of the AQI value, latitude, and longitude.
- Add columns for "Month" and "Year" by parsing your "Date" column (hint: `lubridate` package)
- Hint: the dimensions of this dataset should be 14,752 x 9.

9. Spread your datasets such that AQI values for ozone and PM2.5 are in separate columns. Each location on a specific date should now occupy only one row.

10. Call up the dimensions of your new tidy dataset.
11. Save your processed dataset with the following file name: "EPAair_O3_PM25_NC1819_Processed.csv"

```
#7 Bind the datasets using `rbind`
EPAair_bind <- rbind(EPAair_O3_2018_selected, EPAair_O3_2019_selected,
                     EPAair_PM25_2018_selected, EPAair_PM25_2019_selected)

#8 Wrangle the combined dataset
EPAair_clean <- EPAair_bind %>%
  #filter the sites of interest
  filter (Site.Name %in%
            c("Linville Falls", "Durham Armory", "Leggett", "Hattie Avenue",
  "Clemmons Middle", "Mendenhall School", "Frying Pan Mountain",
  "West Johnston Co.", "Garinger High School", "Castle Hayne",
  "Pitt Agri. Center", "Bryson City", "Millbrook School" )) %>%
  #group by date, site, AQS parameter, and county
  group_by(Date, Site.Name, AQS_PARAMETER_DESC, COUNTY) %>%
  #take the mean of the AQI value, latitude, and longtitude
  filter(!is.na(DAILY_AQI_VALUE) & !is.na(SITE_LATITUDE) & !is.na(SITE_LONGITUDE)) %>%
  summarise(AQI_mean = mean(DAILY_AQI_VALUE),
            latitude_mean = mean(SITE_LATITUDE),
            longitude_mean = mean(SITE_LONGITUDE)) %>%
  #add month and year
  mutate(Month = month(Date), Year = year(Date))
```

```
## `summarise()` has grouped output by 'Date', 'Site.Name', 'AQS_PARAMETER_DESC'.
## You can override using the `.groups` argument.
```

```
#Check dimensions
dim(EPAair_clean)
```

```
## [1] 7899    9
```

```
#9 Spread datasets, AQI values for O3 and PM2.5 are in separate columns
EPAair_tidy <- EPAair_clean %>%
  pivot_wider(names_from = AQS_PARAMETER_DESC,
              values_from = AQI_mean,
              names_prefix = "AQI_")

#10 Check dimensions
dim(EPAair_tidy)
```

```
## [1] 4637    9
```

```
#11 Save the processed dataset
write.csv(EPAair_tidy, row.names = FALSE,
          here("Data", "Processed", "EPAair_O3_PM25_NC1819_Processed.csv"))
```

Generate summary tables

12. Use the split-apply-combine strategy to generate a summary data frame. Data should be grouped by site, month, and year. Generate the mean AQI values for ozone and PM2.5 for each group. Then, add a pipe to remove instances where mean **ozone** values are not available (use the function `drop_na` in your pipe). It's ok to have missing mean PM2.5 values in this result.

13. Call up the dimensions of the summary dataset.

```
#12 Generate a summary data frame
EPAair_summary <- EPAair_tidy %>%
  #grouped by site, month, and year
  group_by(Site.Name, Month, Year) %>%
  #take the mean AQI values
  summarise(Ozone_mean = mean(AQI_Ozone),
            PM25_mean = mean(AQI_PM2.5)) %>%
  #remove the missing ozone values, and keep the missing PM2.5 values
  drop_na(Ozone_mean)
```

```
## `summarise()` has grouped output by 'Site.Name', 'Month'. You can override
## using the `.groups` argument.
```

```
#13 Check the dimensions
dim(EPAair_summary)
```

```
## [1] 112   5
```

14. Why did we use the function `drop_na` rather than `na.omit`? Hint: replace `drop_na` with `na.omit` in part 12 and observe what happens with the dimensions of the summary date frame.

    Answer: While `drop_na()` function removes rows where the specified column contains a missing value, `na.omit()` is a generic function that works on the whole data frame. In this case, we want to remove the rows where Ozone value is missing and keep the ones where PM2.5 value may be missing. `na.omit()` will also drops the rows where `PM25_mean` is NA, which will remove more rows than we expect.